2.1.1 Unit testing

Unit testing is a software testing technique by which individual units are tested to determine that a software component does what it's supposed to do. Tests that are written at the unit level are known for their fast execution because they run in isolation and don't require integrated environments. However, it's important to note that these unit tests only focus on individual methods or components, so they don't verify any integration points. Any dependencies are mocked so the full code and testing coverage are contained within a specific code. Consequently, they also stand the furthest apart from the end-user experience and may not provide a complete validation of system behavior.

2.1.2 Integration testing

To address the limitations of unit testing, it's essential to develop integration tests to assess the interaction between one or two components to other components or services. By validating the integration points between components or services, teams can identify and address any potential problems or inconsistencies that may arise when different parts of the system interact.

2.1.3 End-to-end testing

At the top of the test automation pyramid, we find the E2E tests. E2E tests offer the highest level of confidence because they closely mimic user interactions and replicate real-world scenarios. E2E tests provide comprehensive coverage by testing the entire system, validating the flow and behavior across various components and services. An E2E test is commonly driven from a user interface (UI) and requires all connected services to be deployed to a dedicated environment.

While E2E tests offer the greatest level of confidence, it's important to consider the tradeoffs associated with E2E tests. E2E tests tend to be slower to execute, cost more to develop because they require a dedicated test environment that is similar to production, and involve a higher level of